

Week 02 - Task 5: Prompt Evaluation Report

Objective

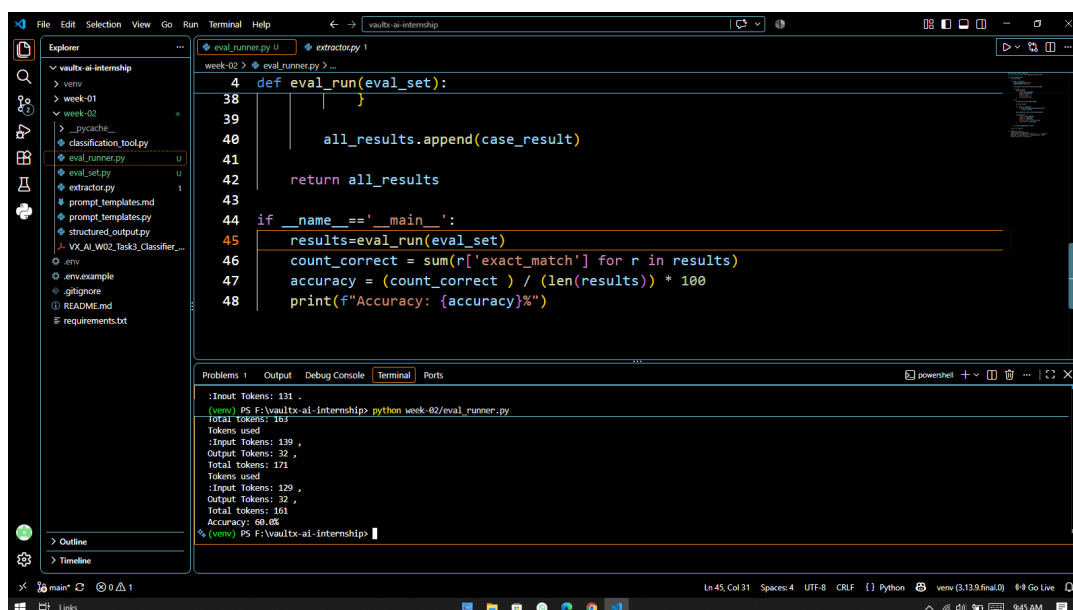
The goal of this task was to measure the accuracy of the support ticket classifier built in Task 3, identify weaknesses in its prompt, iterate on the prompt, and measure the resulting improvement in accuracy.

Evaluation Set

A 15-case evaluation set was created (eval_set.py), each with a customer support message and a manually verified expected classification covering category, priority, sentiment, and needs_human. An eval runner (eval_runner.py) sends each message through the classifier, compares the result to the expected values field-by-field, and reports overall accuracy along with the details of any failing cases.

Baseline Result

Running the original Task 3 prompt against the 15-case eval set produced an accuracy of **60.0%** (9/15 exact matches).



The screenshot shows a VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `eval_runner.py` file with the following code:

```
4 def eval_run(eval_set):
38     ...
39
40     all_results.append(case_result)
41
42     return all_results
43
44 if __name__ == '__main__':
45     results=eval_run(eval_set)
46     count_correct = sum(r['exact_match'] for r in results)
47     accuracy = (count_correct / (len(results))) * 100
48     print(f"Accuracy: {accuracy}%")
```

The terminal at the bottom shows the output of running the script:

```
:Input Tokens: 131 ,
Total Tokens: 163
Tokens used
:Input Tokens: 139 ,
Output Tokens: 32 ,
Total Tokens: 171
Tokens used
:Input Tokens: 129 ,
Output Tokens: 32 ,
Total Tokens: 161
Accuracy: 60.0%
```

Baseline run - 60.0% accuracy

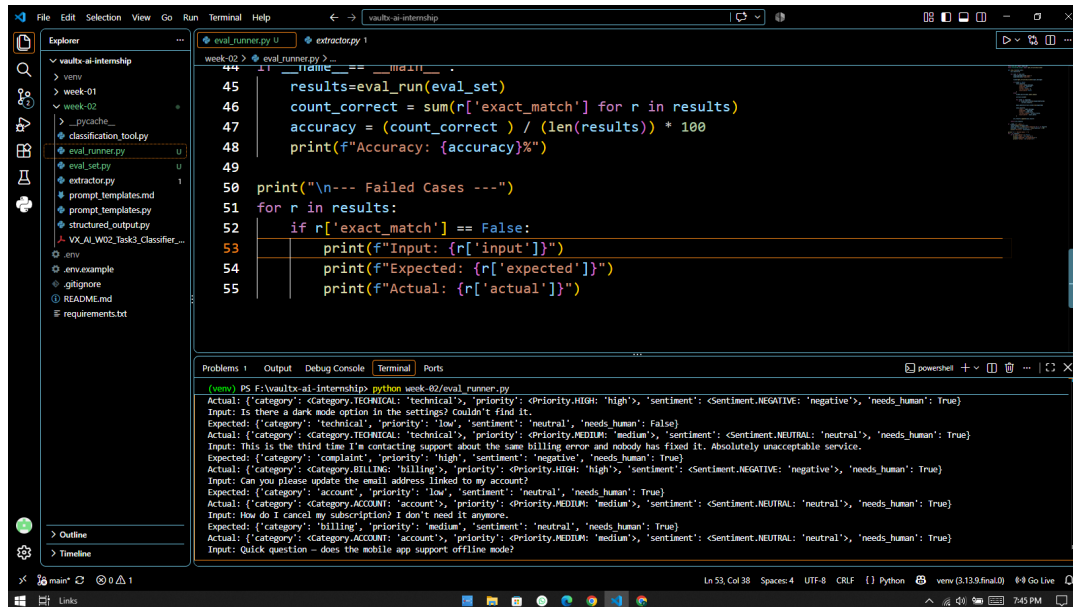
Issues Identified in Baseline

- Category confusion:** The model classified based on surface keywords rather than intent - e.g. a cancellation request was classified as 'account' instead of 'billing', and a repeated-complaint message was classified as 'billing' instead of 'complaint'.
- Priority overestimation:** Routine, non-urgent requests (e.g. updating an email address, a simple feature question) were frequently assigned 'medium' priority instead of 'low'.
- needs_human overused:** Simple informational questions that could be answered like an FAQ were marked as needing human intervention.

Prompt Iteration 1

Added explicit classification rules to the prompt: distinguishing category by customer intent rather than topic keywords, defining priority defaults (low unless urgency or a blocking issue is signaled), and clarifying that needs_human should only be true for actions or non-trivial problems, not simple informational questions.

This raised accuracy from 60.0% to **86.67% (13/15)**. Remaining failures at this stage are shown below.



```
45 results=eval_run(eval_set)
46 count_correct = sum(r['exact_match'] for r in results)
47 accuracy = (count_correct) / (len(results)) * 100
48 print(f"Accuracy: {accuracy}%")
49
50 print("\n--- Failed Cases ---")
51 for r in results:
52     if r['exact_match'] == False:
53         print(f"Input: {r['input']}")
54         print(f"Expected: {r['expected']}")
55         print(f"Actual: {r['actual']}")
```

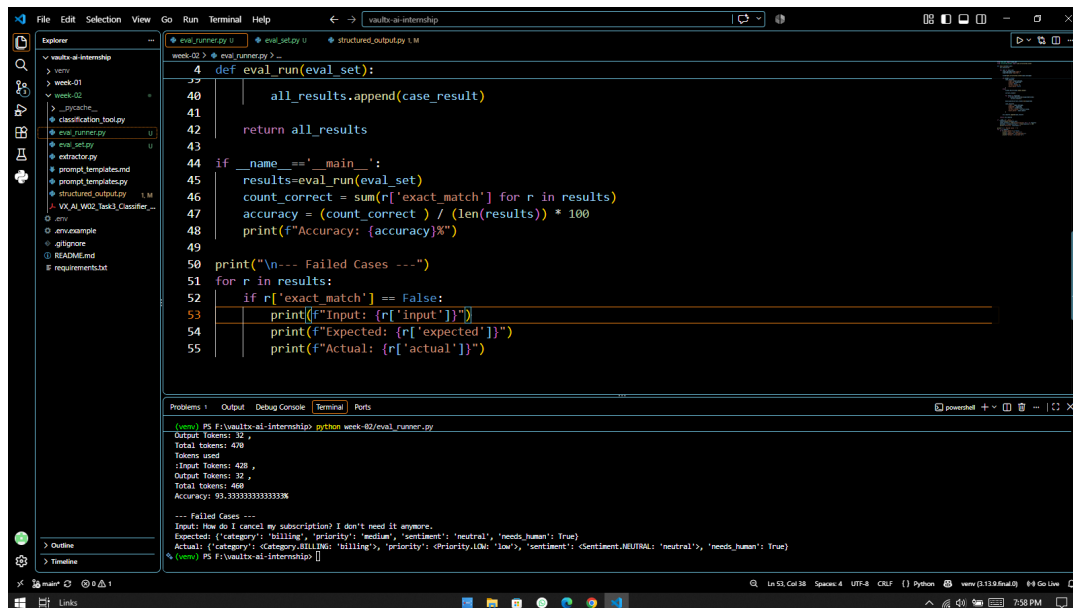
```
Actual: {'category': 'Category.TECHNICAL: 'technical', 'priority': 'priority.HIGH: 'high', 'sentiment': 'Sentiment.NEGATIVE: 'negative', 'needs_human': True}
Input: Is there a dark mode option in the settings? couldn't find it.
Expected: {'category': 'technical', 'priority': 'low', 'sentiment': 'neutral', 'needs_human': False}
Actual: {'category': 'Category.TECHNICAL: 'technical', 'priority': 'priority.MEDIUM: 'medium', 'sentiment': 'Sentiment.NEUTRAL: 'neutral', 'needs_human': True}
Input: This is the third time I'm contacting support about the same billing error and nobody has fixed it. Absolutely unacceptable service.
Expected: {'category': 'complaint', 'priority': 'high', 'sentiment': 'negative', 'needs_human': True}
Actual: {'category': 'Category.BILLING: 'billing', 'priority': 'priority.HIGH: 'high', 'sentiment': 'Sentiment.NEGATIVE: 'negative', 'needs_human': True}
Input: Can you please update the email address linked to my account?
Expected: {'category': 'account', 'priority': 'low', 'sentiment': 'neutral', 'needs_human': True}
Actual: {'category': 'Category.ACCOUNT: 'account', 'priority': 'priority.MEDIUM: 'medium', 'sentiment': 'Sentiment.NEUTRAL: 'neutral', 'needs_human': True}
Input: How do I cancel my subscription? I don't need it anymore.
Expected: {'category': 'billing', 'priority': 'medium', 'sentiment': 'neutral', 'needs_human': True}
Actual: {'category': 'Category.ACCOUNT: 'account', 'priority': 'priority.MEDIUM: 'medium', 'sentiment': 'Sentiment.NEUTRAL: 'neutral', 'needs_human': True}
Input: Quick question - does the mobile app support offline mode?
```

After Iteration 1 - 86.67% accuracy, with remaining failure cases listed

Prompt Iteration 2

Two of the five remaining failures were traced back to inconsistent labels in the eval set itself rather than model error, and were corrected in eval_set.py (e.g. a feature question re-labeled from 'technical' to 'general' to match the new category rule). The priority rule was also refined to account for implicit financial consequences (e.g. an unwanted subscription that will keep charging if not cancelled), since the initial rule only considered explicit urgency language.

This raised accuracy to **93.33% (14/15)**, an improvement of 33.33 percentage points over the baseline.



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'vaults-ai-internship' with various files and folders. The main editor window displays a Python script named 'eval_runner.py'. The script defines a function 'eval_run(eval_set)' which processes an evaluation set and returns results. It also includes a main block that runs the evaluation and prints the accuracy. The terminal at the bottom shows the output of running the script, indicating an accuracy of 93.33333333333333%.

```
4 def eval_run(eval_set):
40     all_results.append(case_result)
41
42     return all_results
43
44 if __name__ == '__main__':
45     results=eval_run(eval_set)
46     count_correct = sum(r['exact_match'] for r in results)
47     accuracy = (count_correct / len(results)) * 100
48     print(f"Accuracy: {accuracy}%")
49
50 print("\n--- Failed Cases ---")
51 for r in results:
52     if r['exact_match'] == False:
53         print(f"Input: {r['input']}")
54         print(f"Expected: {r['expected']}")
55         print(f"Actual: {r['actual']}")
```

```
(venv) PS F:\vaults-ai-internship> python week-02/eval_runner.py
Output Tokens: 32 ,
Total tokens: 478
Tokens used
Input Tokens: 428 ,
Output Tokens: 32 ,
Total tokens: 460
Accuracy: 93.33333333333333%

--- Failed Cases ---
Input: How do I cancel my subscription? I don't need it anymore.
Expected: {'category': 'billing', 'priority': 'medium', 'sentiment': 'neutral', 'needs_human': True}
Actual: {'category': 'category.billing', 'priority': 'priority.low', 'sentiment': 'Sentiment.NEUTRAL: 'neutral'', 'needs_human': True}
(venv) PS F:\vaults-ai-internship>
```

After Iteration 2 - 93.33% accuracy (final)

Remaining Failure Case (Documented, Not Force-Fixed)

One case remains unresolved: 'How do I cancel my subscription? I don't need it anymore.' was expected to be 'medium' priority (due to the financial consequence of an unwanted ongoing charge), but the model assigned 'low' priority, likely because no active harm (e.g. an existing double-charge) was mentioned in the message. This is considered a genuinely subjective edge case rather than a clear model error, and is documented here rather than force-fit with an overly specific rule that risks overfitting the prompt to this single eval case.

Conclusion

Through two rounds of targeted prompt iteration based on eval set failure analysis, classifier accuracy improved from 60.0% to 93.33% on the 15-case evaluation set. This demonstrates the value of building an eval harness before shipping a prompt: failures were diagnosed field-by-field, root causes were identified and separated from eval-set labeling inconsistencies, and improvements were verified quantitatively rather than assumed.